

SECTION 1 — RÔLE, AUTHENTIFICATION ET INSCRIPTION

1.1 Enum UserRole et inscription

✅ **FONCTIONNE** — UserRole.STUDENT existe (models.py:42).

L'endpoint /auth/register (auth.py:102) assigne role = "student" en dur. Le schéma UserCreate (schemas.py:15-22) ne contient PAS de champ role — injection impossible depuis le client.

1.2 Niveau scolaire

✅ **FONCTIONNE** — NiveauScolaire enum complet avec 21 valeurs tunisiennes (models.py:157-190). Présent à l'inscription via UserCreate.niveau_scolaire (schemas.py:22), assigné à la création (auth.py:112). Le seul endpoint de mise à jour utilisateur (users.py:110-142) exige le rôle teacher_or_admin — un étudiant ne peut PAS modifier son propre niveau_scolaire.

1.3 Rattachement à une école

⚠️ **PARTIEL** — Le rattachement se fait UNIQUEMENT via school_name ou school_domain à l'inscription (auth.py:85-101). Si aucun n'est fourni, le premier trouvée est utilisée (auth.py:99). Mécanismes ABSENTS : pas de code d'invitation, pas de sélection manuelle dans l'UI (vérifié dans RegisterPage.tsx), pas d'import CSV côté élève.

1.4 Écran d'onboarding/bienvenue

❌ **ABSENT** — Aucune page onboarding/bienvenue dans le frontend. La recherche "onboarding", "welcome", "bienvenue" ne retourne que des résultats sans rapport (composant WelcomeMessage dans LearnerAIChatPage.tsx qui est le message de bienvenue du tuteur IA, pas un onboarding). Après inscription, l'élève est redirigé directement vers /dashboard.

1.5 Vérification d'identité (verify-identity)

⚠️ **PARTIEL** — L'endpoint POST /learner/verify-identity (learner.py:937-966) existe mais est ORPHelin pour les élèves : il est réservé aux comptes independent_paid (teachers) uniquement (learner.py:948). Aucun appel frontend pour un étudiant. L'endpoint GET /learner/verify-identity/status (learner.py:974) est aussi présent mais sans frontend student.

SECTION 2 — ACCÈS AUX COURS ET RBAC

2.1 has_course_access() câblé

✅ **FONCTIONNE** — has_course_access() est importé (learner.py:21) et appelé à 4 endroits : lignes 301 (syllabus), 411 (détail leçon), 576 (quiz start), 621 (quiz questions). Chaque appel lève HTTP 403 si l'accès est refusé. La logique vérifie : gratuité → auteur → inscription → accès école → pack individuel → pack école (course_access.py:20-85).

2.2 purchase_course déduit le solde

✅ **FONCTIONNE** — purchase_course (courses.py:310-388) vérifie current_user.dt_balance < course.price (ligne 334), puis débite current_user.dt_balance -= course.price (ligne 341). Crée une Transaction (ligne 344) et une CourseEnrollment (ligne 374). Reste du solde retourné dans la réponse.

2.3 Test: élève sans pack sans accès cours payant

✅ **FONCTIONNE** — Le test test_course_access.py vérifie que sans pack/purchase, has_course_access() retourne False pour un cours payant. La logique course_access.py:20-85 retourne False si aucun des 6 tests (gratuit, auteur, inscription, école, pack individuel, pack école) n'est satisfait.

2.4 Filtre multi-tenant (TenantMixin)

✅ **FONCTIONNE** — Le filtre est un event listener sur Session.do_orm_execute (db/session.py). Le contexte est défini dans get_current_user (auth.py:62-74). Les tests test_tenant_filter.py vérifient l'isolation entre école A et école B (8 tests). Le catalogue filtre visibility != "school_only" (catalog.py:78, learner.py:259).

2.5 Faille GET /catalog/courses/{slug}

✅ **CORRIGÉE** — Le endpoint public (catalog.py:72-122) filtre Course.visibility != "school_only" (ligne 78) ET Course.status == CourseStatus.PUBLISHED (ligne 77). Un cours school_only n'est plus exposé par slug.

2.6 Routes frontend protégées par RequireRole

✅ **FONCTIONNE** — Les routes student dans App.tsx (lignes 191-225) utilisent RequireRole roles=[["student", "admin_school"]] pour : /dashboard/courses, /dashboard/courses/:courseId, /dashboard/ai-tutor, /dashboard/wallet, /dashboard/tier. La route /dashboard/inbox (ligne 188) est accessible à tout utilisateur authentifié (pas de RequireRole spécifique).

SECTION 3 — PACKS CÔTÉ ÉLÈVE

3.1 Catalogue de packs côté élève

⚠ PARTIEL — Backend fonctionnel : GET /packs (packs.py:30-78) retourne les packs publiés. Frontend : StudentTierPage.tsx a un lien "Voir les packs" qui pointe vers /dashboard/courses (ligne 201-205), PAS vers une page packs dédiée. La recherche "PackPage" ne trouve aucun composant frontend dédié au listing/achat de packs par l'élève. Le bouton "Voir les packs" redirige vers le catalogue de cours, pas vers les packs.

3.2 Distinction pack inclus via école vs pack à acheter

✅ FONCTIONNE — Le champ `already_included_by_school` est calculé côté backend (packs.py:73) et retourné pour chaque pack. Si l'école de l'élève a déjà un pack actif pour le même niveau, `already_included_by_school=true`.

3.3 Double achat inutile bloqué

✅ FONCTIONNE — Le guard dans packs.py (vérifié via `test_pack_access.py`) empêche l'achat d'un pack déjà inclus par l'école. Le champ `already_included_by_school` est disponible pour le frontend afin de masquer le bouton d'achat.

3.4 Accès dynamique après ajout de cours au pack

✅ FONCTIONNE — `has_course_access()` vérifie la correspondance `course.niveau_scolaire == pack.niveau_scolaire` et `course.category` in `pack.matières` à chaque appel (`course_access.py:88-101`). Pas de cache figé — un cours ajouté au pack est immédiatement accessible.

3.5 Champ `owner_type` sur StudyPack

✅ FONCTIONNE — `StudyPack.owner_type` existe (models.py:746) avec valeurs "school" | "eduai_catalog". Le champ `school_id` FK est aussi présent (models.py:747).

SECTION 4 — WALLET DE CRÉDITS IA

4.1 Distinction des 4 poches

✅ FONCTIONNE — `WalletPool` enum contient `TRIAL`, `SCHOOL_ALLOCATED`, `PURCHASED`, `SUBSCRIPTION` (models.py). La consommation suit la priorité : `SUBSCRIPTION` → `SCHOOL_ALLOCATED` → `TRIAL` → `PURCHASED` (le plus périssable en premier, wallet.py:135-139).

4.2 GET /wallet/balance et /wallet/history

✅ FONCTIONNE — GET /wallet/balance (wallet.py:19-50) retourne les soldes par poche avec dates d'expiration. GET /wallet/history (wallet.py:53-88) retourne l'historique paginé des transactions.

4.3 Alertes de solde faible (20%/10%)

✅ FONCTIONNE — check_low_balance_alert() (wallet.py:217-285) calcule le pourcentage par rapport à la référence, retourne "warning" à 20% et "critical" à 10%. Pour les students : message "Solde faible. Contactez votre école" (ligne 269). Un mécanisme de déduplication empêche le spam (ligne 244-254).

4.4 Consommation IA débite le wallet

✅ FONCTIONNE — Tous les endpoints IA (/ai/ask, /ai/explain, /ai/quiz, /ai/correct, /ai/generate, /ai/exercises) appellent consume_credits() avant l'appel IA. Le pré-check de solde est fait avant (wallet_estimate_cost). En cas d'erreur IA, les crédits sont remboursés (refund dans chaque endpoint).

SECTION 5 — TEST DE POSITIONNEMENT ET RECOMMANDATION

5.1 Modèles PlacementTest/PlacementTestResult

✅ FONCTIONNE — Les modèles existent (models.py — recherche confirmée). Le router placement.py expose GET /placement/tests, GET /placement/tests/{id}, POST /placement/tests/{id}/submit. Le calcul adaptatif retourne 3 niveaux : debutant (<40%), intermédiaire (40-70%), avancé (≥70%).

5.2 Déclenchement automatique à l'activation d'un pack

❌ ABSENT — Aucun code ne déclenche automatiquement un test de positionnement lors de l'activation d'un pack Excellence/Établissement. Le test est accessible manuellement via l'endpoint, mais il n'y a pas de workflow automatique.

5.3 get_recommended_path() et cohérence niveau

✅ FONCTIONNE — get_recommended_path() (recommendation.py:24-31) retourne un parcours différencié selon le palier. Le parcours Découverte affiche les cours inscrits (recommendation.py:49-74), Excellence affiche les matières les plus faibles (recommendation.py:145-181), Établissement affiche le programme national (recommendation.py:241-266).

5.4 GET /learner/daily-objective et get_student_tier()

✅ FONCTIONNE — get_student_tier() (student_tier.py:24-58) calcule dynamiquement le palier depuis les PackPurchase. get_daily_objective() (recommendation.py:34-42) retourne des

objectifs différenciés. Pour Découverte : objectif basique "continuez la leçon" (recommandation.py:77-138). Le test `test_tier_restrictions.py` vérifie que Découverte ne reçoit pas d'objectifs personnalisés via `generate_daily_goal` (retourne None).

SECTION 6 — SYSTÈME D'OBJECTIFS MULTI-HORIZONS

6.1 Modèle LearningGoal et enums

✅ **FONCTIONNE** — LearningGoal model existe (models.py).

Enums GoalHorizon (daily/weekly/monthly/quarterly/annual), GoalStatus (on_track/behind/completed/missed), GoalMetricType (5 métriques), GoalSource (4 sources) sont tous définis. La table `learning_goals` est créée par migration SQL.

6.2 Statut recalculé dynamiquement

✅ **FONCTIONNE** — `compute_goal_status()` (goal_tracking.py:202-257) calcule la valeur actuelle via `_compute_current_value()` qui interroge LessonProgress, QuizAttempt, etc. Le statut n'est JAMAIS stocké — il est calculé à chaque appel. Le

test `test_goal_tracking.py::test_status_recalculates_dynamiquement` vérifie ce comportement.

6.3 Génération auto daily/weekly et assignation quarterly/annual

✅ **FONCTIONNE** — `generate_daily_goal()` (2 leçons/jour, goal_tracking.py:264-302), `generate_weekly_goal()` (5h/semaine, goal_tracking.py:305-342). L'assignation trimestriel se fait via POST /pedagogical-lead/goals/quarterly (goals.py:256-323) qui applique à TOUTE une classe. L'annuel via POST /learner/goals/annual (goals.py:185-239).

6.4 Calendrier scolaire tunisien

⚠️ **PARTIEL** — `school_calendar.py` contient les dates par défaut pour 2025-2026 (T1: 15 sept–15 déc, T2: 5 jan–31 mar, T3: 1 av–15 juin). Les dates ne sont PAS ajustées au bulletin officiel du Ministère — le fichier comporte un avertissement explicite (school_calendar.py:8-10 : "Ces dates par défaut doivent être ajustées selon le bulletin officiel").

6.5 Messages de statut (ton constructif)

✅ **FONCTIONNE** — Messages définis

dans `STATUS_MESSAGES` (goal_tracking.py:29-34) :

- `on_track` : "Tu es sur la bonne voie !"
- `behind` : "Un peu de retard, mais rien d'impossible à rattraper."
- `completed` : "Objectif atteint — félicitations !"

- missed : "Objectif non atteint cette fois — voici comment repartir." Tous constructifs, aucun culpabilisant.
-

SECTION 7 — DEVOIRS, QUIZ, CERTIFICATION

7.1 Flux complet devoir assigné → soumission → correction → retour

⚠ PARTIEL — Le flux existe dans `lms.py` : `create_assignment` (ligne 36), `submit_assignment` (ligne 144). Cependant, la soumission (`submit_assignment`) retourne un résultat basique (`SubmissionResult`). La correction automatisée n'est pas implémentée — pas de scoring automatique, pas de feedback détaillé retourné à l'élève. Le flux est fonctionnel pour la soumission mais la correction est manuelle/incomplète.

7.2 Soumission de devoir/quiz alimente le calcul des objectifs

✅ FONCTIONNE — `compute_goal_status()` interroge `QuizAttempt` pour `QUIZ_AVERAGE_SCORE` (`goal_tracking.py:109-116`) et `LessonProgress` pour `LESSONS_COMPLETED` (`goal_tracking.py:93-106`). Les quiz complétés via `submit_quiz_attempt` (`learner.py:651`) créent un `QuizAttempt` avec `score_percent` qui alimente directement le calcul des objectifs.

7.3 Génération de certificat en fin de cours

✅ FONCTIONNE — `GET /learner/courses/{course_id}/certificate` (`learner.py:810-853`) génère le certificat si `enrollment.status == "completed"`. Les données sont correctement peuplées : `student_name=user.full_name` or `user.email`, `course_name=course.title`, `certificate_number=CERT-{random}`, `verification_code=secrets.token_hex(16)`. Pas de valeurs de test oubliées.

SECTION 8 — AI TUTOR ET DIFFÉRENCIATION PAR PALIER

8.1 POST `/ai/ask`, `/ai/explain`, `/ai/quiz` — réponses RAG

✅ FONCTIONNE — Les 3 endpoints utilisent `RAGService` (`ai.py:119, 187, 271`) qui interroge l'index FAISS de l'école. `ask_tutor` charge l'historique de conversation pour le contexte (`ai.py:152`). Les réponses sont basées sur le contenu indexé de l'école, pas génériques.

8.2 `get_ai_feature_level()` et application

✅ FONCTIONNE — `get_ai_feature_level()` (`student_tier.py:61-71`) retourne "basic" pour Découverte, "adaptive" pour Excellence, "curriculum_aligned" pour Établissement. Application réelle :

- `/ai/exercises` (`ai.py:318-324`) : bloque "basic" → HTTP 403

- `/ai/generate` (ai.py:546-552) : bloque si `≠ "curriculum_aligned"` → HTTP 403
Un élève Découverte a réellement moins de fonctionnalités IA.

8.3 Alignement RTL/LTR automatique

✅ **FONCTIONNE** — Le composant `LearnerAIChatPage.tsx` détecte la langue via `detectInputDirection()` (ligne 50-56) qui compte les caractères arabes vs latins. Les messages sont affichés avec `dir={textDir}` (ligne 456) et `style={{ textAlign }}` (ligne 457). Le champ `detected_language` est sauvegardé dans chaque message (ai.py:75-80). Le placeholder s'adapte aussi (arabe vs français, ligne 543).

SECTION 9 — BOÎTE DE RÉCEPTION, NOTIFICATIONS, INTERACTION PARENT

9.1 GET `/api/inbox/messages`

✅ **FONCTIONNE** — L'endpoint (inbox.py:14-74) retourne les messages directs + broadcasts correspondant au rôle de l'utilisateur. Filtrage par `school_id` pour les non-super_admins. Types de messages : directs (`receiver_id`), broadcasts (`target_audience`). Marquage comme lu via `PUT /inbox/messages/{id}/read`.

9.2 Compte/rôle "parent"

❌ **ABSENT** — La recherche dans `models.py` pour "parent" ne retourne que "the parent course" (commentaire dans `learner.py:568`). L'enum `UserRole` ne contient PAS de rôle "parent". Aucune trace d'un rôle parent dans tout le backend. Le design UX mentionne l'interaction parent mais elle n'est pas implémentée.

9.3 Permissions parent

❌ **ABSENT** — Puisque le rôle parent n'existe pas, il n'y a aucune permission associée.

9.4 Écran "Ce que voit ton parent"

❌ **ABSENT** — Aucun composant frontend ne correspond à cette fonctionnalité. La recherche dans les fichiers TSX ne retourne rien de pertinent.

9.5 Notifications rappel/objectif et félicitations

⚠️ **PARTIEL** — `generate_soft_notifications()` (notifications.py:18-94) génère 3 types de notifications : `daily_reminder`, `weekly_celebration`, `quarterly_celebration`. Cependant : (1) le mécanisme de déduplication est un TODO (notifications.py:105 : # TODO: implémenter un vrai stockage), (2) les notifications sont générées à la volée mais jamais persistées en base (pas de table `notification_log`), (3) aucun canal de diffusion (in-app, email, push) n'est câblé — les notifications sont juste retournées par l'endpoint mais jamais envoyées.

SECTION 10 — FRONTEND : NAVIGATION ET EXPÉRIENCE

10.1 Structure de navigation à 5 zones

⚠ PARTIEL — La navigation réelle dans App.tsx (lignes 191-225) contient : Catalogue (/dashboard/courses), Devoirs (/dashboard/assignments), Tuteur IA (/dashboard/ai-tutor), Portefeuille (/dashboard/wallet), Mon Palier (/dashboard/tier). Ce n'est PAS une architecture à 5 zones (Accueil/Parcours/IA/Objectifs/Profil) — c'est une structure différente avec des chemins directs. Pas de composant "Objectifs" dans la navigation principale (les objectifs sont dans /dashboard/tier).

10.2 Écran d'accueil réel : objectif du jour visible ?

✗ ABSENT — Le dashboard student (StudentDashboard.tsx) affiche : header, 3 stats (hardcodées à 0), et 5 liens rapides (Catalogue, Devoirs, Tuteur IA, Portefeuille, Mon Palier). L'objectif du jour n'est PAS visible en premier. Les stats sont statiques ("0" pour toutes les valeurs) — pas de données réelles affichées.

10.3 Adaptation de densité d'interface selon niveau scolaire

✗ ABSENT — L'interface est identique pour tous les niveaux. Aucune adaptation de taille de police, d'espacement, ou de densité d'information selon que l'élève est en primaire ou au secondaire.

10.4 Sélecteur de langue et effet RTL/LTR

⚠ PARTIEL — Le sélecteur de langue n'existe que dans le tuteur IA (LearnerAIChatPage.tsx) via la détection automatique de la langue. Il n'y a PAS de sélecteur de langue dans le profil élève. Le basculement RTL/LTR fonctionne dans le chat IA mais le reste de l'interface est uniquement en LTR (français).

10.5 Gamification excessive

✅ PAS DE PROBLÈME — La recherche "gamif", "mascot", "streak", "classement", "points" ne retourne aucun élément de gamification excessive. Le seul "badge" trouvé est dans AdminLayout.tsx (un compteur de notifications, pas un badge gamifié). Aucune mascotte, aucun classement public, aucun système de points superflu.

SECTION 11 — SÉCURITÉ SPÉCIFIQUE AU RÔLE STUDENT

11.1 Test: student ne peut accéder aux endpoints teacher/admin

✅ FONCTIONNE — Les endpoints critiques sont protégés :

- create_assignment (lms.py:42) : vérifie _is_admin_or_teacher(current_user)
- update_assignment (lms.py:103) : idem

- delete_assignment (lms.py:128) : idem
- create_course (admin_school) : protégé par require_school_admin
- Les tests test_role_rejection.py (13 tests) vérifient le rejet pour chaque combinaison rôle/endpoint.

11.2 Student ne peut pas modifier son propre niveau_scolaire/rôle

✅ **FONCTIONNE** — L'endpoint update_user (users.py:110) exige require_teacher_or_admin (ligne 114). Un student ne peut PAS l'appeler. La modification de rôle est further restreinte aux admins (users.py:131-136). Il n'existe PAS d'endpoint self-service de mise à jour du profil pour les students.

11.3 Cohérence de casse des rôles frontend

✅ **FONCTIONNE** — Les routes frontend utilisent des rôles en MAJUSCULES ("student", "admin_school", "teacher" dans App.tsx). La vérification dans RequireRole compare avec .toUpperCase() (App.tsx:163-164). Le store auth retourne le rôle tel que stocké en base (minuscule). La cohérence est maintenue via la comparaison insensible à la casse.

TABLEAU RÉCAPITULATIF

❌ **ABSENT** (5 points)

Point	Description
1.4	Écran d'onboarding/bienvenue post-inscription
5.2	Déclenchement automatique du test de positionnement à l'activation d'un pack
9.2	Rôle "parent" dans UserRole
9.3	Permissions parent (lecture seule bilans, messages d'encouragement)
9.4	Écran "Ce que voit ton parent" (transparence)
10.2	Objectif du jour visible en premier sur le dashboard

Point	Description
10.3	Adaptation de densité d'interface selon niveau scolaire

⚠ PARTIEL (8 points)

Point	Description
1.3	Rattachement école : uniquement via school_name/domain, pas d'invitation
1.5	verify-identity : orphelin pour students (que pour teachers)
3.1	Catalogue packs : backend OK, frontend redirige vers cours (pas page packs)
6.4	Calendrier tunisien : dates par défaut, pas ajustées au bulletin officiel
7.1	Flux devoir : soumission OK, correction automatisée absente
9.5	Notifications : générées mais jamais persistées ni envoyées
10.1	Navigation : structure différente de l'architecture 5 zones prévue
10.4	Langue : détection auto dans chat, pas de sélecteur global

✅ FONCTIONNE (34 points)

1.1, 1.2, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 3.2, 3.3, 3.4, 3.5, 4.1, 4.2, 4.3, 4.4, 5.1, 5.3, 5.4, 6.1, 6.2, 6.3, 6.5, 7.2, 7.3, 8.1, 8.2, 8.3, 9.1, 11.1, 11.2, 11.3, 10.5 (pas de gamification excessive)

TOP 5 DES LACUNES CRITIQUES

1. Dashboard student inutilisable (10.2) — Les stats sont hardcodées à 0, l'objectif du jour n'est pas visible. L'élève voit un écran vide après connexion.
2. Rôle parent totalement absent (9.2-9.4) — L'interaction parent, élément clé du design UX e-learning tunisien, n'est pas implémentée du tout.

3. Notifications jamais diffusées (9.5) — Le système génère des notifications mais ne les stocke ni ne les envoie. C'est un endpoint mort.
4. Pas d'onboarding (1.4) — L'élève est jeté directement dans le dashboard après inscription sans guidance.
5. Dashboard stats figées (10.2) — "0 Mes Cours", "0 AI Tokens", "0 DT Balance" — les vraies données ne sont pas chargées.

POINTS NON VÉRIFIABLES (?)

Point	Raison
3.4 (accès dynamique après ajout cours)	Testé par lecture de code (pas de cache figé) mais pas vérifié avec un scénario réel d'ajout de cours mid-pack
8.1 (qualité des réponses RAG)	Impossible de tester la qualité sans base de données FAISS peuplée et fournisseur IA actif
11.1 (complet)	Les 13 tests test_role_rejection.py vérifient les principaux endpoints, mais la liste complète des endpoints teacher/admin n'est pas exhaustive dans les tests